



An End-to-End Automated ML Pipeline in Snowflake

Maegan Kornexl, Solutions Engineer

Sept. 23, 2026

Safe Harbor and Disclaimers

Other than statements of historical fact, all information contained in these materials and any accompanying oral commentary (collectively, the “Materials”), including statements regarding (i) Snowflake’s business strategy, plans or priorities, (ii) Snowflake’s new or enhanced products, services, and technology offerings, including those that are under development or not generally available, (iii) market growth, trends, and competitive considerations, (iv) our vision for Snowpark, the Data Cloud, and industry-specific Data Clouds, including the expected benefits and network effects of the Data Cloud; and (v) the integration, interoperability, and availability of Snowflake’s products, services, or technology offerings with or on third-party platforms or products, are forward-looking statements. These forward-looking statements are subject to a number of risks, uncertainties and assumptions, including those described under the heading “Risk Factors” and elsewhere in the Annual Reports on Form 10-K and the Quarterly Reports on Form 10-Q that Snowflake files with the Securities and Exchange Commission. In light of these risks, uncertainties, and assumptions, the future events and trends discussed in the Materials may not occur, and actual results could differ materially and adversely from those anticipated or implied in the forward-looking statements. As a result, you should not rely on any forwarding-looking statements as predictions of future events. Forward-looking statements speak only as of the date the statements are first made and are based on information available to us at the time those statements are made and/or management’s good faith belief as of that time. Except as required by law, we undertake no obligation, and do not intend, to update the forward-looking statements in these Materials.

Any future product or roadmap information (collectively, the “Roadmap”) is intended to outline general product direction. The Roadmap is not a commitment, promise, or legal obligation for Snowflake to deliver any future products, features, or functionality; and is not intended to be, and shall not be deemed to be, incorporated into any contract. The actual timing of any product, feature, or functionality that is ultimately made available may be different from what is presented in the Roadmap. The Roadmap information should not be used when making a purchasing decision. In case of conflict between the information contained in the Materials and official Snowflake documentation, official Snowflake documentation should take precedence over these Materials. Further, note that Snowflake has made no determination as to whether separate fees will be charged for any future products, features, and/or functionality which may ultimately be made available. Snowflake may, in its own discretion, choose to charge separate fees for the delivery of any future products, features, and/or functionality which are ultimately made available.

The Materials may contain information provided by third-parties. Snowflake has not independently verified this information, and usage of this information does not mean or imply that Snowflake has adopted this information as its own or independently verified its accuracy.

Who is this webinar for?

Anyone who is:

- New to Snowflake ML
- Knows enough Snowflake ML, but wants a more automated pipeline
- Curious about best practices
- Likes listening to people talk about ML



Where Are Today's Materials?

(This link is also in the chat)

https://github.com/sfc-gh-mkornexl/E2E_ML_HOL

This session is also being recorded, and will be available on-demand in ~2 weeks here:

<https://www.snowflake.com/en/webinars/?tags=status%2Fon-demand>



A Note About Account Permissions

- Some of the items in today's lab will be inaccessible to you based on your specific role's permissions in your environment.
 - This means if you try to run those cells without those permissions, they'll error out on you- sorry in advance!
- HOWEVER, the vast majority of this lab was specifically designed to avoid this issue, so barring any smaller features I introduce here, you should be okay.
 - If a majority of the cells in the lab aren't running for you, figure out who at your company controls your Snowflake and go ask them please!!!!

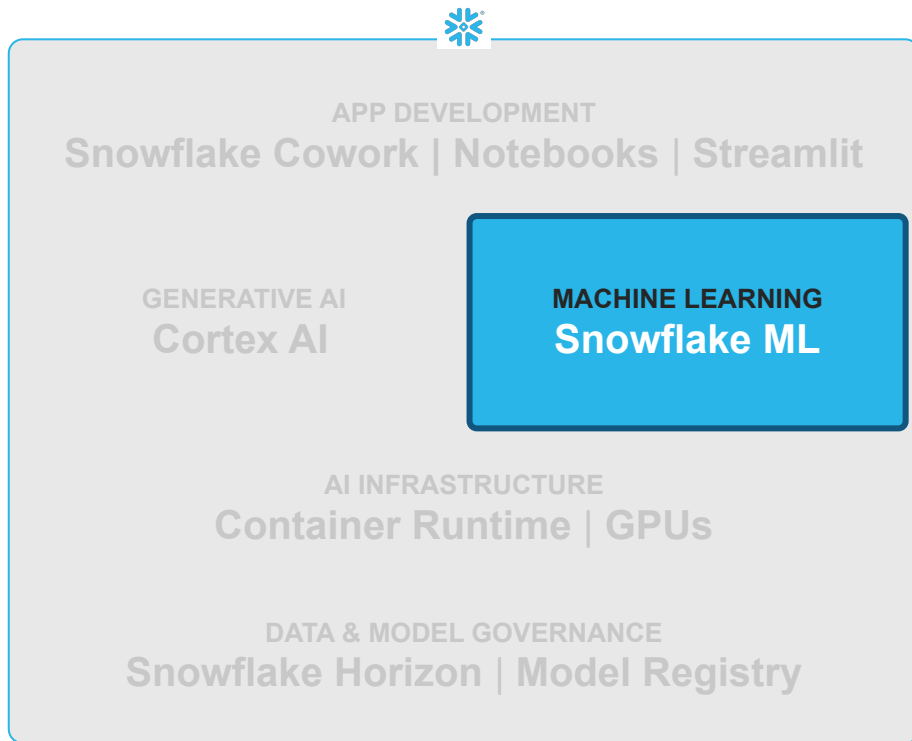


Who can I contact with Questions about the lab?

- Questions **about the lab**: me, at maegan.kornexl@snowflake.com
- Questions about an ML pipeline **NOT covered in the lab**: your Snowflake account team
 - I can also help you *find* your account team if you don't know them, but I *can't* help you with your own environment's specific deployments.
- Please still ask those questions- you aren't a bother!- but I may not be the right person to help you with custom deployments.



Unified Platform for End-to-End Gen AI & ML



Predictive ML Still Drives Massive Business Value



Predict Demand

Create sales and other types of consumption forecast to optimize business plans



Recommend Products

Uplift revenue per customer with personalized recommendations



Identify Anomalies

Catch risks such as fraud or errors for timely intervention



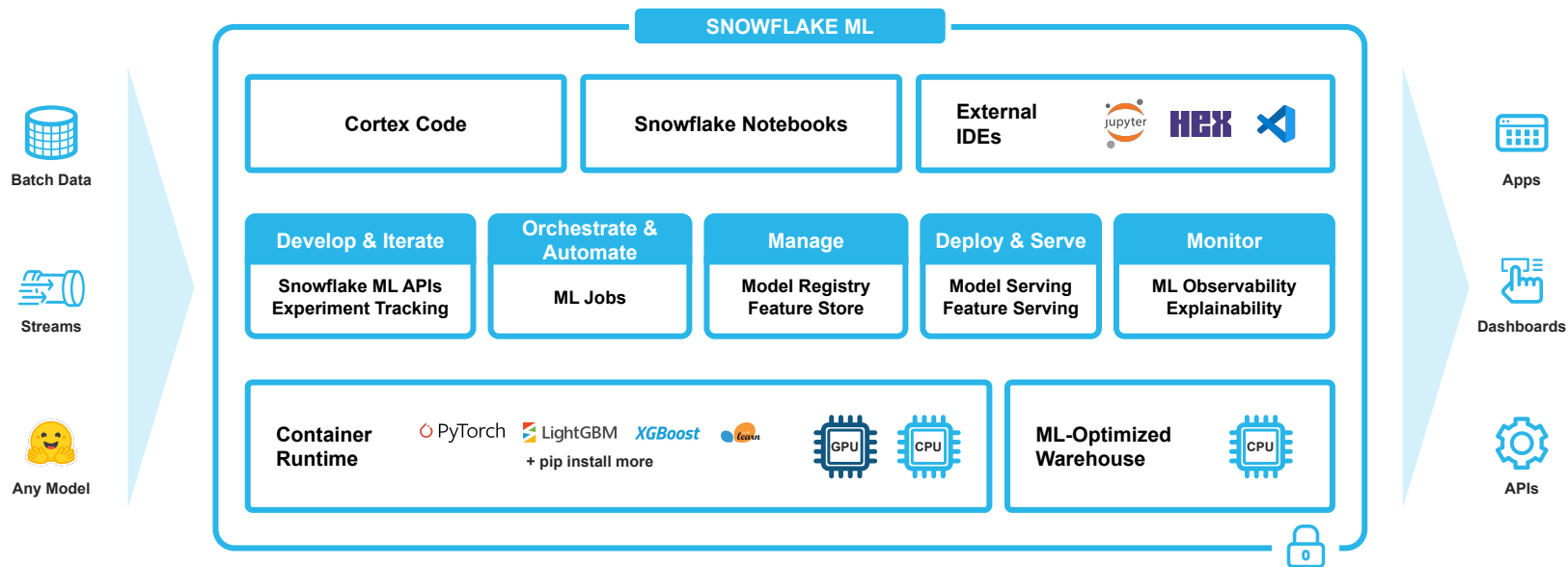
Improve Prioritization

Use budget and resources more efficiently with lead scoring and other types of prioritization



Snowflake ML: Comprehensive, Flexible, Integrated

Agentic ML tooling drives 10x productivity for DS and MLEs



Snowflake Notebooks

Familiar Jupyter experience

Get the power of a Jupyter notebook directly connected to the governed Snowflake data

Inside Snowflake Workspaces

Organize and run notebooks alongside your other files

Powerful for AI/ML

Runs in our pre-built Container Runtime with fully-managed access to CPUs and GPUs, parallel data loading, and distributed training APIs

Governed collaboration

Enable users to collaborate via Git-backed Workspaces or Shared Workspaces, which give RBAC collaboration with change history

The screenshot displays the Snowflake Notebooks interface. On the left is a sidebar with a file explorer showing a directory structure including 'my_utils', 'demo_1_feature_high...', 'demo_3_gpu_example...', '2nd_notebook.ipynb', 'packages', 'click-no-more-demo...', 'README.md', 'Untitled.sql', 'diamonds', and 'nb-examples-main'. The main area shows a Jupyter notebook titled '0.start_here.ipynb' with a code cell containing Python code for training a BERTopic model. Below the code cell is a table of variables.

1. Train BERTopic Model on GPU resources

The diamonds dataset has been widely used in data science and machine learning. We will use it to demonstrate Snowflake's native data science transformers in terms of database functionality and Spark & Pandas compatibility, using non-synthetic and statistically appropriate data that is well known to the ML community.

```
1 # %time
2 topic_model = BERTopic()
3 tppic, probs = topic_model.fit_transform(sample_docs, embedding)
```

Import Libraries

```
1 # Snowpark for Python
2 from snowflake.snowpark.types import DoubleType
3 import snowflake.snowpark.functions as F
4 topic_model = BertTopic
```

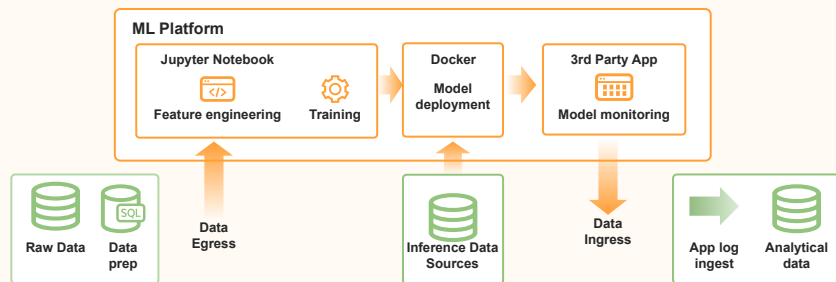
Name	Type	Shape	Preview
session	Session	-	<snowflake.snowpark.session.Session object at 0x7faadabbe0>
diamonds_df	DataFrame	(53940, 11)	row carat cut color clarity ... table price x y z n 1 0.23 Ideal E SI...



Snowflake ML: Unified Platform for Data and AI

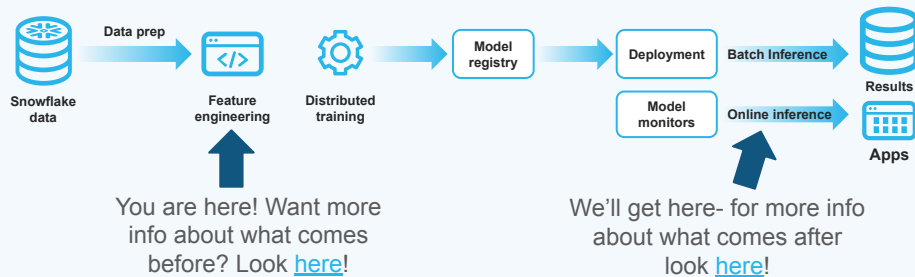
Single platform. No data movement. Unified governance.

Traditional Approach



- ✗ Multiple, disparate tools
- ✗ Complex engineering processes to orchestrate
- ✗ Inefficient data movement
- ✗ Complex governance

Snowflake ML



- ✓ Streamlined processes
- ✓ Efficient and performant
- ✓ Integrated with the Data Platform
- ✓ Simplified security and governance



Most Common Predictive ML Use Cases



Recommendation Systems

Product Recommendation

Customer Targeting

Next Best Action



Embedding Models

Product Recommendation

RAG

Similarity Search

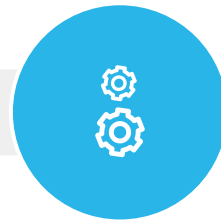


Computer Vision

Anomaly detection for defects

Image classification for e-commerce

OCR analysis for document processing



Large Scale Tree Models

Demand Forecasting

Churn Prediction

Lifetime Value Models



So, What Are We Building Today?



Model Lifecycle in Snowflake



UNIFIED GOVERNANCE



**Develop
& Iterate**



**Orchestrate
& Automate**



Manage



**Deploy
& Serve**



Monitor



Model Lifecycle in Snowflake



UNIFIED GOVERNANCE



**Develop
& Iterate**

**Migrate ML
Development with:**

- Snowflake Notebooks
- Distributed ML APIs
- Cortex Code
- Experiment tracking



**Orchestrate
& Automate**



Manage



**Deploy
& Serve**

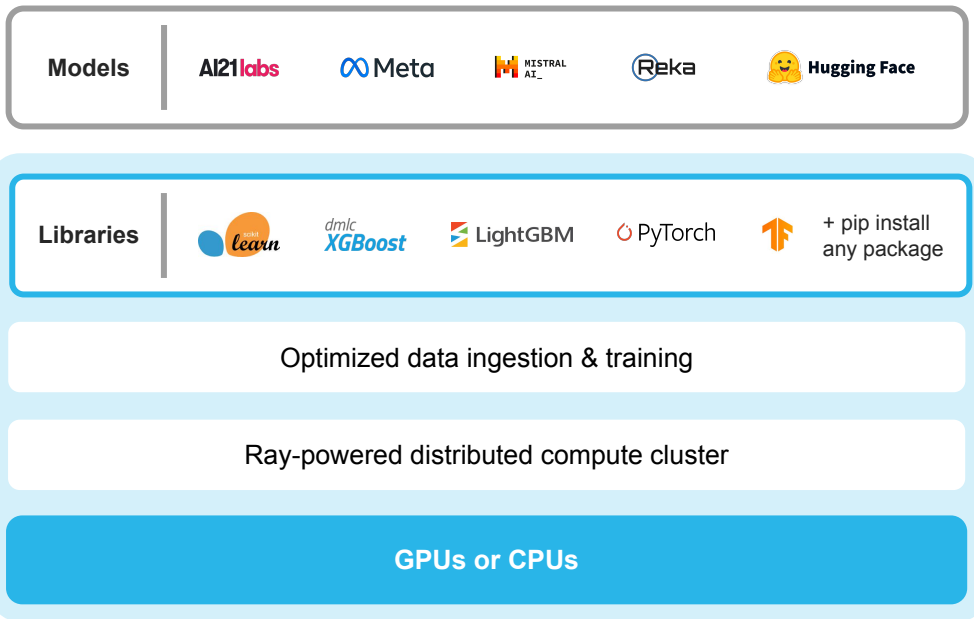


Monitor



Container Runtime: Flexible & Scalable Development

- Pre-built container environment with latest ML packages.
- Easy access to CPU and GPU compute pools
- Optimized data ingestion from tables or stages
- Modern ML infrastructure with Ray to facilitate distributed compute at scale

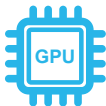


Scale Up Development with Training Enhancements



Distributed Training APIs

Distributed model training and hyperparameter tuning leveraging multiple cores / GPUs across the entire cluster



GPU-accelerated Scikit-learn

Container runtime come preloaded with NVIDIA libraries that accelerate pandas and scikit-learn algorithms on GPUs



Partition many model training and inference

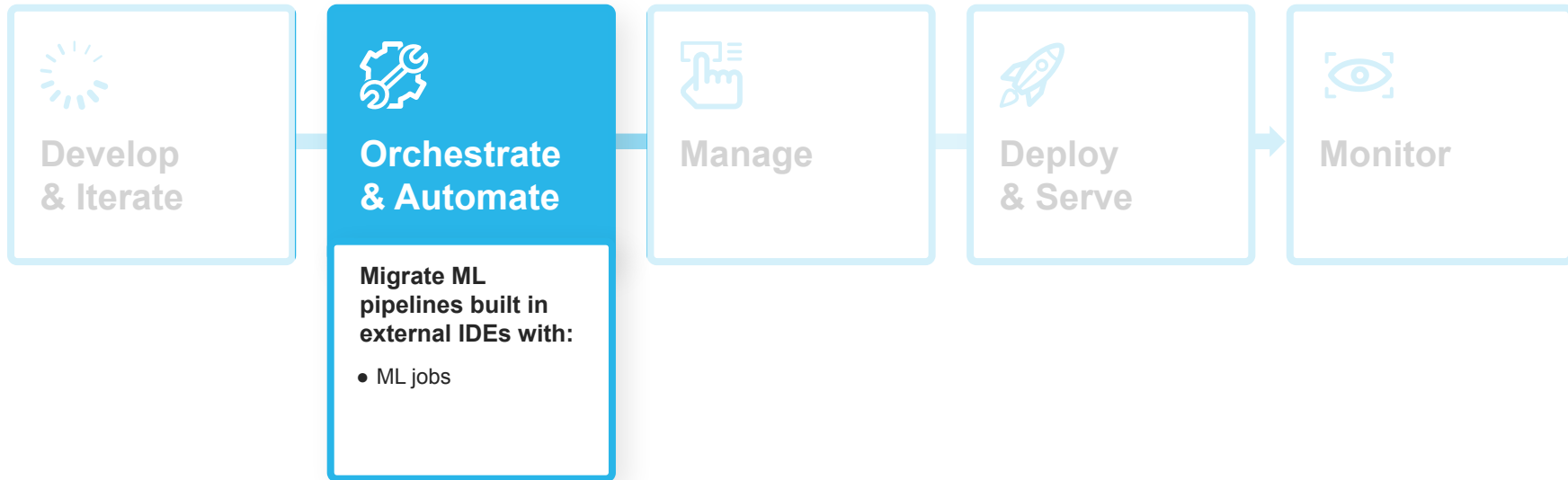
Built-in solution for fitting multiple models in parallel on partitions of data automatically



Model Lifecycle in Snowflake



UNIFIED GOVERNANCE



Real-Time Feature & Model Serving

Low-latency predictions

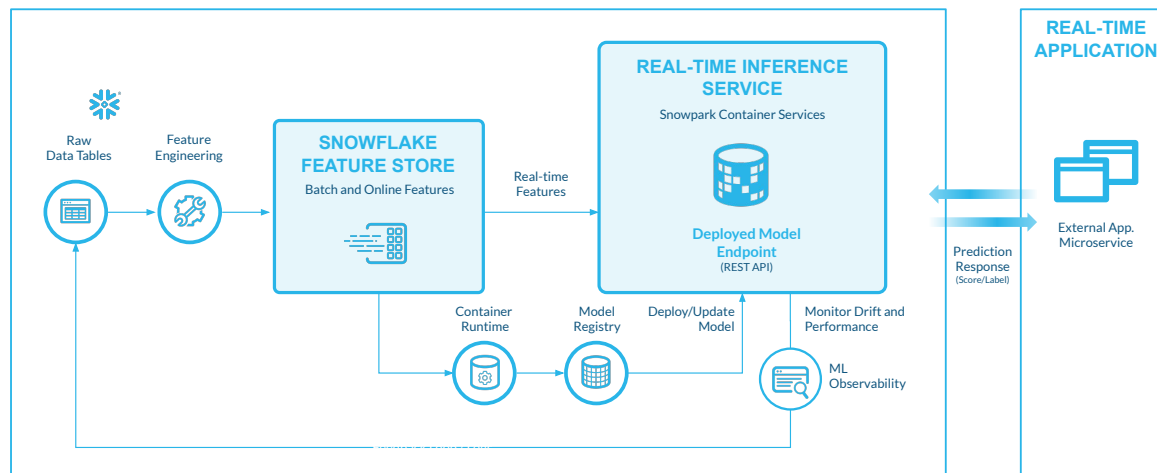
Serve features in under 30 ms and models in under 100 ms to power online use cases such as personalized recommendations and fraud detection.

Natively integrated MLOps

Easily monitor, explain, and receive alerts on performance and accuracy over time

Unified security and governance

Online and batch inference on the same platform as your governed data



Snowflake Feature Store: Discover, Manage, Govern Features Alongside your Data

Build and Manage ML Features In One Place

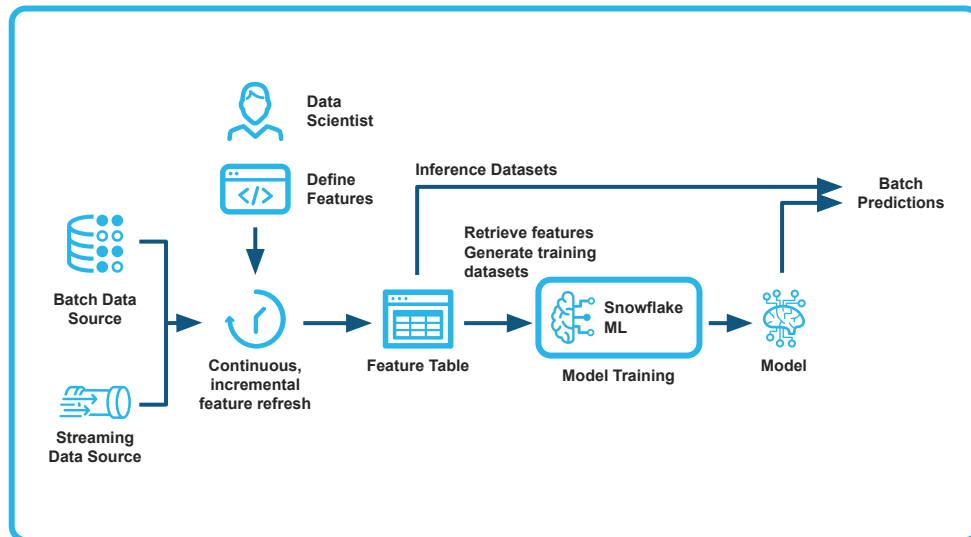
Define, maintain and serve features used for ML training and inference

Easily collaborate

Promote discoverability, reuse, and governance of features

Automate Feature Computation Pipelines

Automated, continuous maintenance of features from batch and streaming sources



ML Experiment Tracking: Enhanced Manageability

Easily Identify Best Performing Model

Visualize & Compare your model versions as you iterate during experimentation

Accelerated Dev to Prod

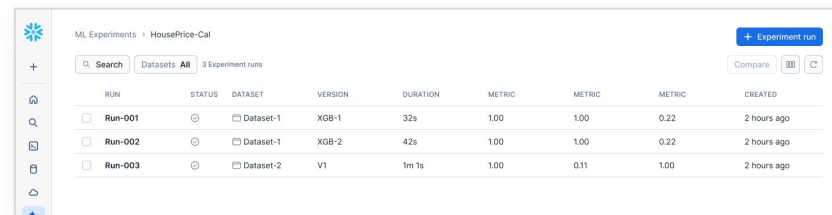
Organize your experiments and easily find all the relevant information to deploy a model to production

Seamless Reproducibility

Reproduce experiments to mitigate issues found in production and build on top of past experiments

Easy Collaboration

Collaborate with team members by sharing run details such as metadata, results, artifacts



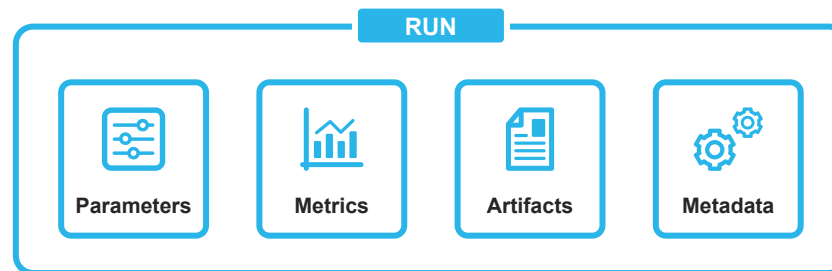
ML Experiments > HousePrice-Cal

+ Experiment run

Search Datasets All 3 Experiment runs

RUN	STATUS	DATASET	VERSION	DURATION	METRIC	METRIC	METRIC	CREATED
☐ Run-001	⊙	Dataset-1	XGB-1	32s	1.00	1.00	0.22	2 hours ago
☐ Run-002	⊙	Dataset-1	XGB-2	42s	1.00	1.00	0.22	2 hours ago
☐ Run-003	⊙	Dataset-2	V1	1m 1s	1.00	0.11	1.00	2 hours ago

Compare [] [C]



Snowflake Model Registry: Manage All Your Models in One Place

Log Models Built Anywhere

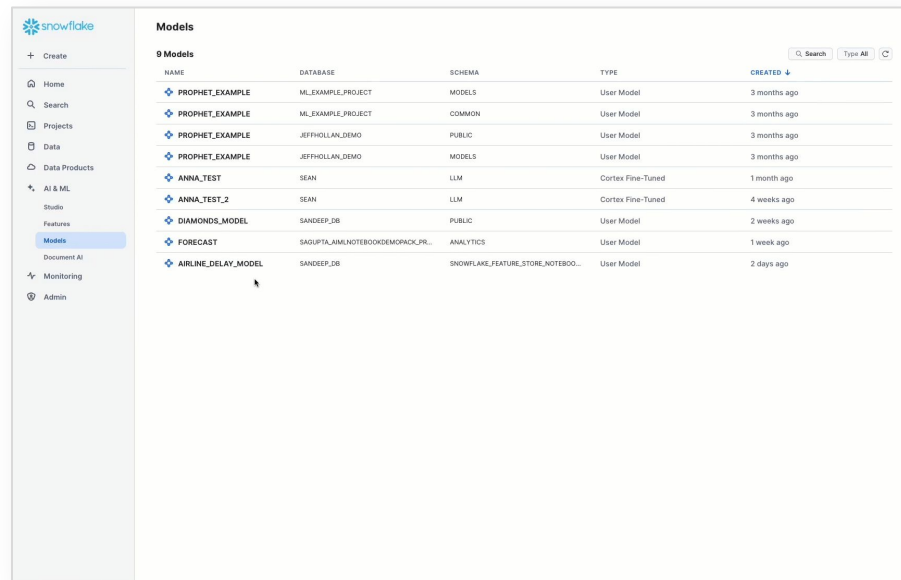
Log models trained anywhere for inference in Snowflake

Strong governance controls

Manage models, versions, and metadata with full role-based access controls.

More scale and ease of use

- Now supports logging and using models with >500 inputs.
- pip package dependencies for support beyond Snowflake Conda (Warehouse deployment).

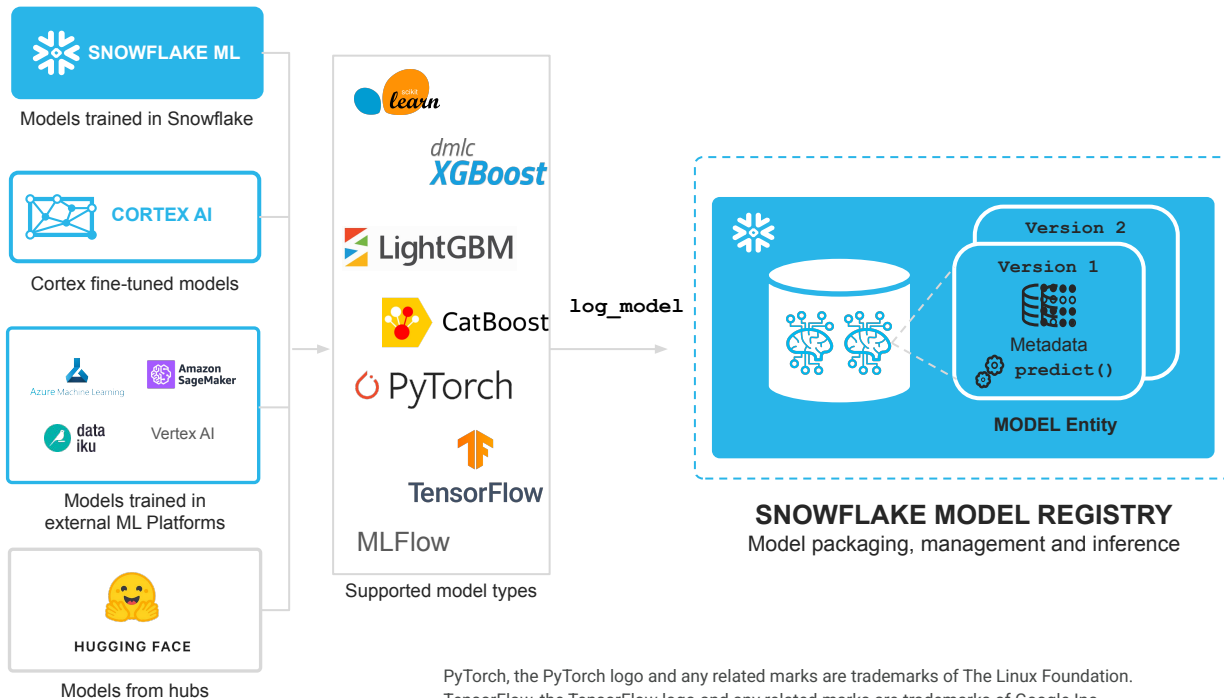


The screenshot shows the Snowflake Model Registry interface. On the left is a sidebar with navigation options: Create, Home, Search, Projects, Data, Data Products, AI & ML, Studio, Features, Models (highlighted), Document AI, Monitoring, and Admin. The main panel displays a table of 9 models. The table has columns for NAME, DATABASE, SCHEMA, TYPE, and CREATED. Each row represents a model, with some names truncated. The models are listed in descending order of creation time.

NAME	DATABASE	SCHEMA	TYPE	CREATED
PROPHET_EXAMPLE	ML_EXAMPLE_PROJECT	MODELS	User Model	3 months ago
PROPHET_EXAMPLE	ML_EXAMPLE_PROJECT	COMMON	User Model	3 months ago
PROPHET_EXAMPLE	JEFFHOLLAN_DEMO	PUBLIC	User Model	3 months ago
PROPHET_EXAMPLE	JEFFHOLLAN_DEMO	MODELS	User Model	3 months ago
ANNA_TEST	SEAN	LLM	Cortex Fine-Tuned	1 month ago
ANNA_TEST_2	SEAN	LLM	Cortex Fine-Tuned	4 weeks ago
DIAMONDS_MODEL	SANDEEP_DB	PUBLIC	User Model	2 weeks ago
FORECAST	SAGUPTA_AIMLNOTEBOOKDEMOBACKPR...	ANALYTICS	User Model	1 week ago
AIRLINE_DELAY_MODEL	SANDEEP_DB	SNOWFLAKE_FEATURE_STORE_NOTEBOD...	User Model	2 days ago



Snowflake Model Registry: Manage All Your Models in One Place



PyTorch, the PyTorch logo and any related marks are trademarks of The Linux Foundation.
TensorFlow, the TensorFlow logo and any related marks are trademarks of Google Inc.



ML Jobs: Automate ML Pipeline Management

Build Anywhere, Execute in Snowflake

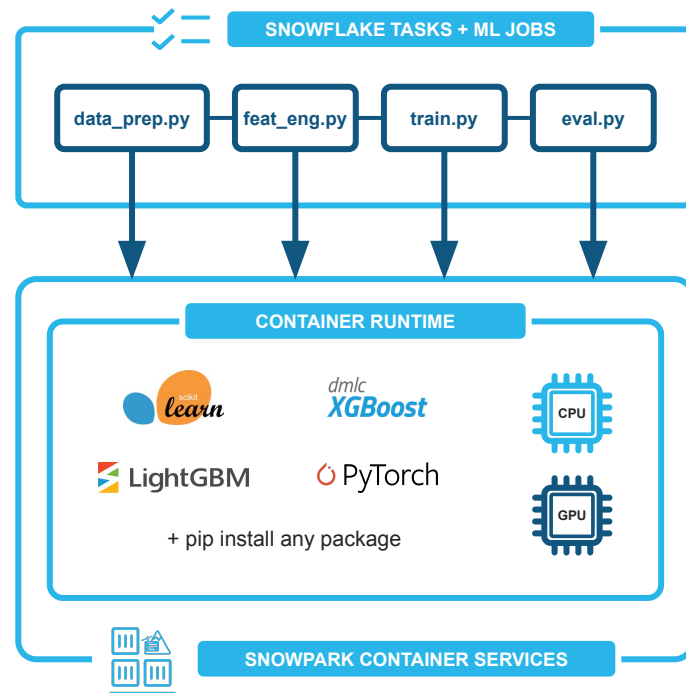
Execute code remotely as an ML Job directly from any IDE (e.g. VS Code) or pipeline orchestrator

Scalability without Data Movement

Run ML workloads at scale with GPUs or distributed APIs using decomposable steps without data movement

Automate with CI/CD

Configure automatic retraining on a schedule or event triggers using Snowflake Tasks. Monitor and view the DAGs with Snowflake Task Graphs



Model Lifecycle in Snowflake



UNIFIED GOVERNANCE



Develop
& Iterate



Orchestrate
& Automate



Manage



Deploy
& Serve

**Bring models built
anywhere to run
inference in
Snowflake with:**

- Model Serving
- Feature Store



Monitor



Model Serving: Scalable Batch and Low Latency Inference

Scalable batch inference

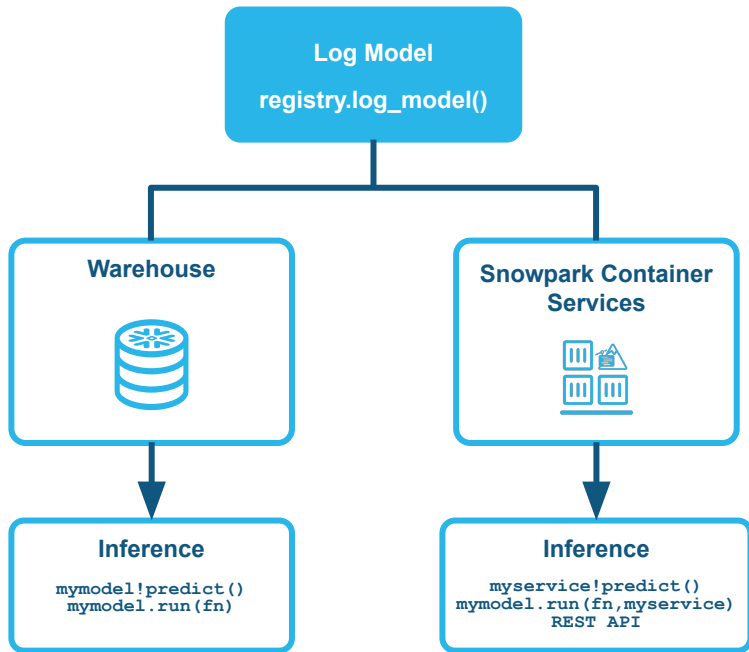
- **Warehouse:**
Distributed inference for smaller models, composing with SQL processing pipelines, and ease of use
- **Snowpark Container Service (SPCS):**
Larger models, GPU-based inference

Low latency inference endpoints

Easily deploy models to SPCS based REST endpoints for low-latency inference

No infra to manage

Log models in the Model Registry and create an inference service with a simple API



Model Lifecycle in Snowflake



UNIFIED GOVERNANCE



Develop
& Iterate



Orchestrate
& Automate



Manage



Deploy
& Serve



Monitor

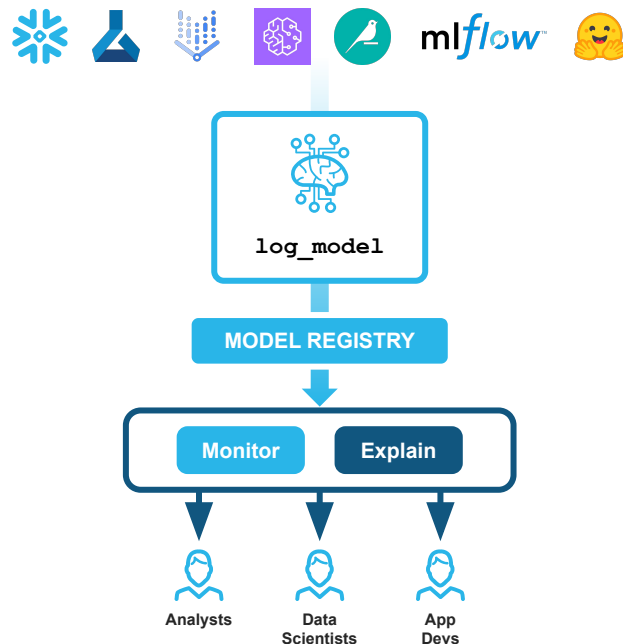
**Monitor models
built anywhere in
production in
Snowflake with:**

- ML Observability
- ML Explainability



ML Observability: Explain and Monitor Model Behavior

- Monitor models in production to ensure more reliable predictions over time
- Setup custom metrics to be monitored along with OOTB metrics
- Generate alerts to take action when models breaches metric thresholds
- Query monitoring results to create your own dashboards
- Automatically Compute Shapley explainability values for any logged model in Model Registry



Model Lifecycle in Snowflake



UNIFIED GOVERNANCE

Govern models in Snowflake with:

- ML Lineage
- Synthetic Datasets
- Data Quality Monitoring



Develop
& Iterate



Orchestrate
& Automate



Manage



Deploy
& Serve



Monitor



Snowflake's Best-in-Class Governance Extends to ML



Data and ML Lineage (GA)

Trace end-to-end lineage of ML artifacts from source data to features, datasets, and models



Horizon: Data Quality Monitoring (GA)

Out-of-the-box system metrics (such as null count) or custom metrics for data that customers can define and automatically measure



Synthetic Datasets (GA)

Build models while maintaining the privacy of sensitive data sets or to easily generate new data to enrich training



Data and ML Lineage

WHAT IS IT

Capability to trace end-to-end lineage of ML artifacts from source data to features, datasets, and models

WHY USE IT

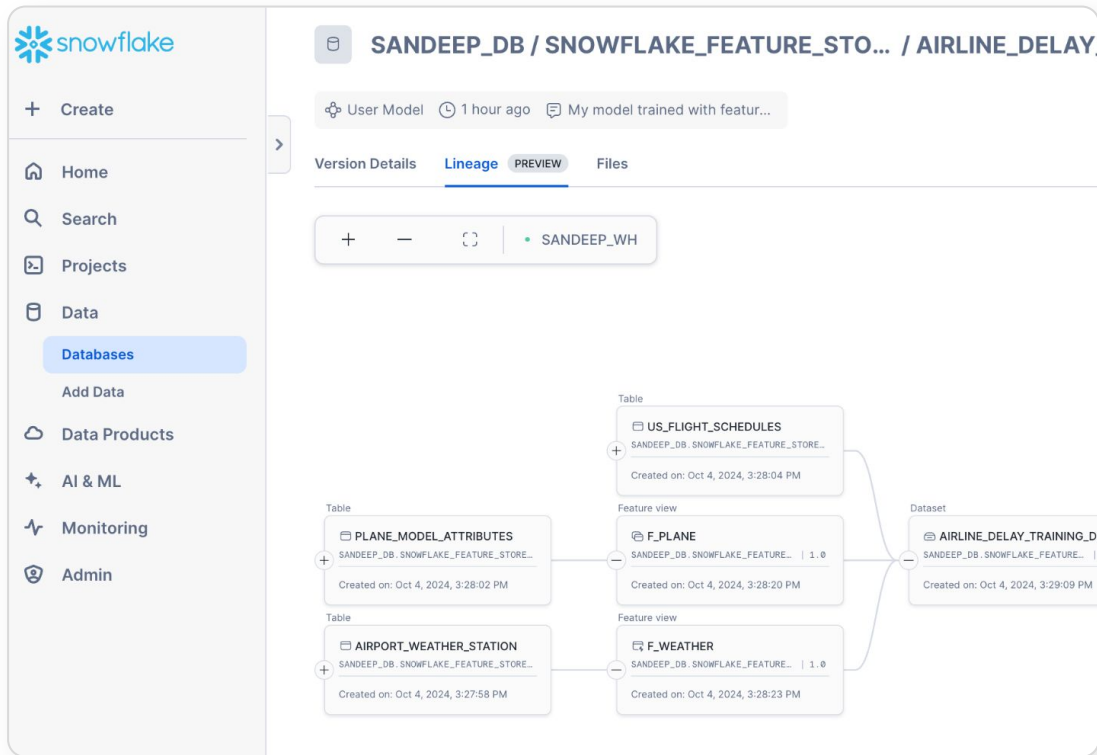
Enable reproducibility, compliance, and debugging across the full lifecycle of ML assets

Example tasks:

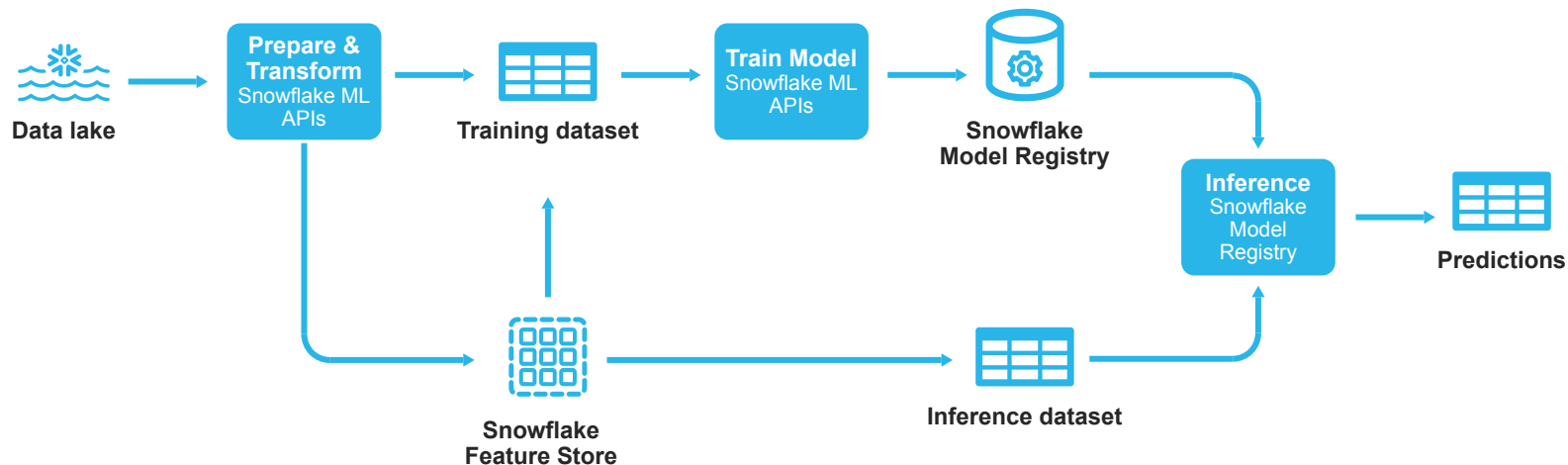
- Compare performance of two model versions
- Trace flow of sensitive data

HOW TO USE IT

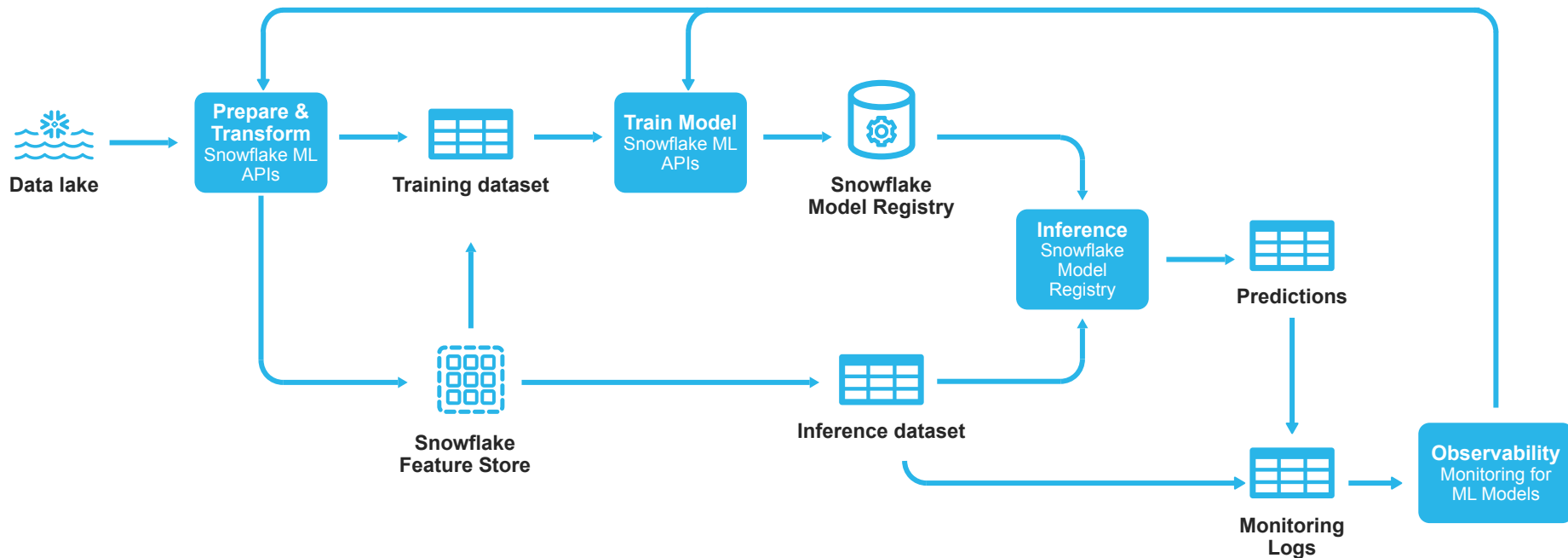
Access built-in UI (PuPr) in Snowsight or analyze via code with Snowflake ML Python APIs



End-to-End ML Workflow with Snowflake ML



End-to-End ML Workflow with Snowflake ML



Online Feature Serving

Support for online features

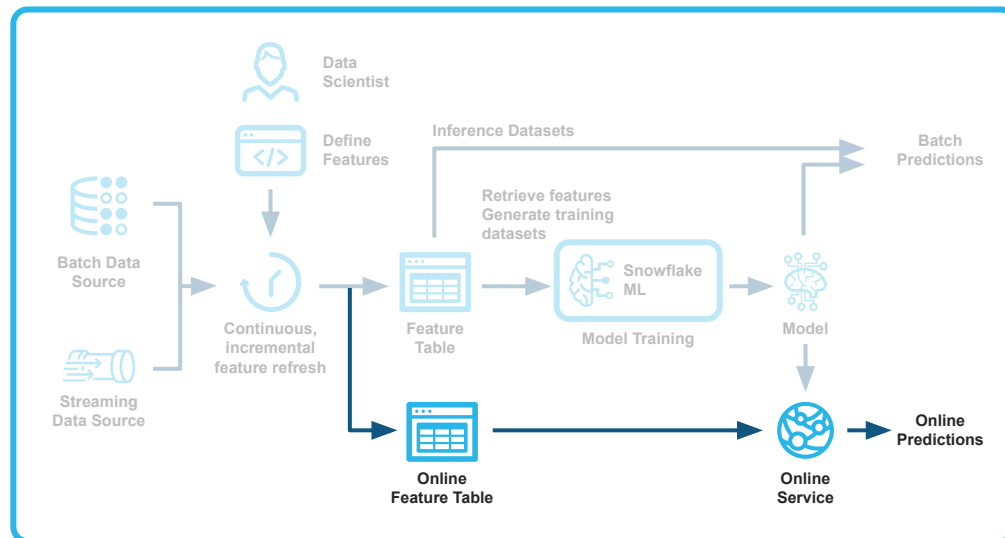
Low-latency serving of ML features for online use cases along with batch in a unified feature store

Automated consistency

Automatic synchronization with offline feature pipeline

High availability & concurrency

100s of QPS and high availability with no infrastructure to build or maintain



Snowflake ML Functions

WHAT IS IT

SQL functions that perform predictive analytics using machine learning under the hood

WHY USE IT

Generate predictions and insights on your structured data with the power of ML—but without its complexity

HOW TO USE IT

Use SQL function in Snowsight or expose as a custom app via Streamlit

ML Functions

`FORECAST ( )`

`ANOMALY_DETECTION ( )`

`TOP_INSIGHTS ( )`

`CLASSIFICATION ( )`



Structured Data



Snowflake ML Functions

SQL functions that perform predictive analytics using ML under the hood



Forecasting

Build more reliable time series forecasts with automated handling of seasonality, missing values and more

GA



Anomaly Detection

Identify outliers and trigger alerts or help find unlikely-to-happen-again situations that should be excluded from analysis

GA



Top Insights

Quickly identify drivers contributing to the change of a given metric across user defined time intervals

GA



Classification

Sort data into different classes (e.g., loyal customer vs. likely to churn) using supervised learning techniques

GA



Who can I contact with Questions about the lab?

- Questions **about the lab**: me, at maegan.kornexl@snowflake.com
- Questions about an ML pipeline **NOT covered in the lab**: your Snowflake account team
 - I can also help you *find* your account team if you don't know them, but I *can't* help you with your own environment's specific deployments.
- Please still ask those questions- you aren't a bother!- but I may not be the right person to help you with custom deployments.





THANK YOU!!!

I hope y'all had a great time!

Appendix



Model Development Deep Dive



Container Runtime for ML: Key Features & Benefits

Addresses key capabilities in the ML development lifecycle

Container Runtime

Pre-configured Packages

- ✓ Comprehensive set of OSS and Snowflake packages to power all stages of ML development
- ✓ Extensibility to be able to leverage custom repositories and packages from hubs such as PyPi and Hugging Face

GPU Pools

- ✓ 1-click configuration to enable GPU access in a Notebook
- ✓ Built-in observability for improved resource allocation and cost management

Training & Hyperparameter Tuning APIs

- ✓ Familiar interfaces of xgboost, lightgbm, pytorch
- ✓ Bayesian-based tuning for optimal results
- ✓ Distributed processing over CPUs or GPUs
- ✓ Easily scale and up down to optimize for cost and performance

Data Connector APIs

- ✓ Parallel data loading across all available CPUs or GPUs
- ✓ Optimal partition sizes leading to better performance
- ✓ Ingestion designed to saturate network



Training Multiple Models on Data Partitions

WHAT IS IT

Built-in solution for fitting multiple models in parallel on partitions of data automatically

WHY USE IT

- **Speed up and parallelize** multiple model training and inference; No need to manually partition data or write UDTFs
- **Build hyper-personalized and individualized models on partitions of data**, such as custom forecasting models for each user/store/product etc

HOW TO USE IT

Write your model code as a standard function

Use Model Registry to log and run the model on partitioned data as a table function

```
Python
class ExampleForecastingModel(custom_model.CustomModel):

    @custom_model.inference_api
    def predict(self, input: pd.DataFrame) -> pd.DataFrame:
        # All data in the partition will be loaded in the input
        # dataframe.
        # ... implement model logic here ...
        return output_df

my_model = ExampleForecastingModel()

# Log model in the registry as a Table Function Model
reg = Registry(session=sp_session, database_name="ML",
               schema_name="REGISTRY")
model_version = reg.log_model(my_model,
                              model_name="my_model",
                              version_name="v1",
                              options={"function_type": "TABLE_FUNCTION"},
                              conda_dependencies=["scikit-learn"],
                              sample_input_data=train_features
                              )

# Invoke model fit/prediction
self.model_version.run(
    input_df,
    function_name="PREDICT",
    partition_column="PARTITION_COLUMN"
)
```



MLOps

Deep Dive



Snowflake Feature Store

WHAT IS IT

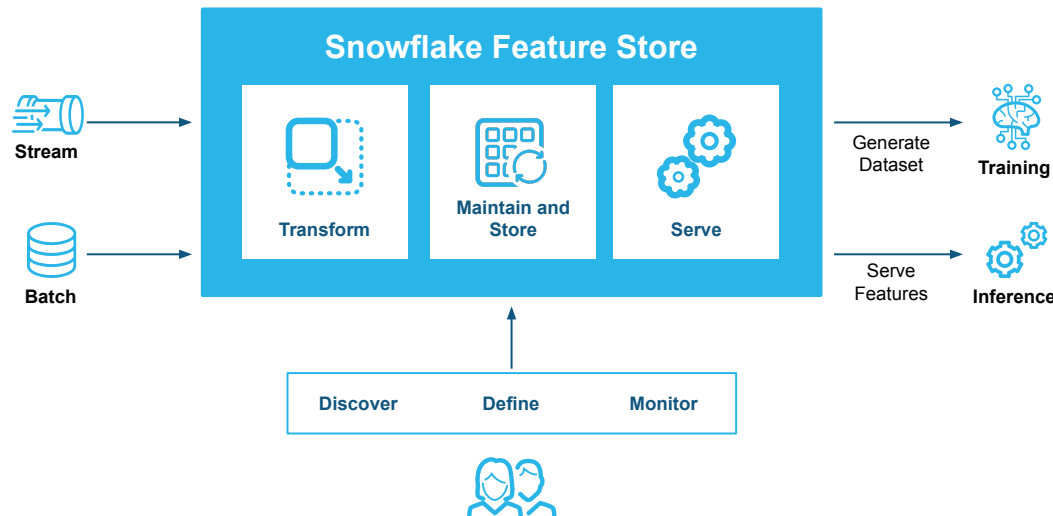
Integrated solution for defining, maintaining and serving features used for machine learning training and inference

WHY USE IT

- Automated, continuous maintenance of features from batch and streaming sources
- Easy feature authoring
- Point-in-time correct feature retrieval with ASOF JOIN
- Promote discoverability, reuse, and governance of features

HOW TO USE IT

Interact via Python SDK in Snowflake ML or from Snowsight UI



Snowflake Model Registry: Key Benefits



Unified Model Catalog

- All common model formats
- Custom models
- LLMs and fine tuned models
- Externally trained models



Flexibility

- Store model metadata and metrics
- Manage model versions
- Use Tags and Aliases



Inference Ready

- Models are inference-ready: No deployment step
- Use from SQL or Python
- Use in Warehouse or on SPCS (with access to GPUs)



Governance

- Fine-grained RBAC for model owners and model usage
- Manage model life cycle and operationalize models from dev to production
- Built-in end to end Lineage



Collaboration

- Securely share models within and across accounts
- Copy models from
- Manage model life cycle and operationalize models from dev to production
- Built-in end to end Lineage



Snowflake Model Registry: Example Use Cases



Train xgboost or pytorch model on CPU or GPU. Run batch inference on CPU in Warehouse or GPU on SPCS



Train many-models on partitions of data. Run large scale, parallel partitioned model inference on thousands of models in minutes



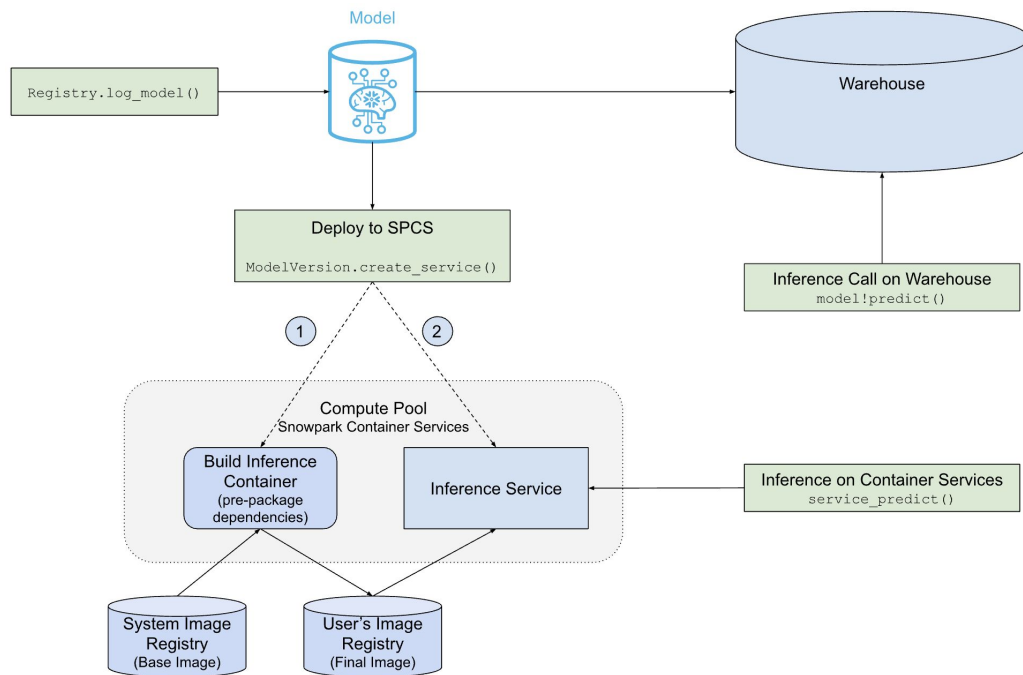
Deploy your externally trained models in Snowflake and run inference at scale without any data movement



Deploy any Hugging Face model or pipeline and run large scale inference on CPUs or GPUs (eg. Embedding models, LLMs)

How Does Model Serving in SPCS Work

- `log_model()` in Model Registry
- `create_service()` API to deploy the model to SPCS; Requires a compute pool and an image repo
- Snowflake will create a model so image and launch SPCS service to run inference.
User does not need to deal with container and Docker details
- User invokes the service from REST API or from service functions from SQL or Python
- **New:** Service will Auto-Suspend when not in use and REST API is not enabled (default 30 mins, configurable)



Example of Model Serving in SPCS

```
# Log the model with pip dependencies
pip_model = reg.log_model(
    embed_model,
    model_name="sentence_transformer_minilm",
    version_name="pip",
    sample_input_data=input_data, # Needed for determining signature of the model
    pip_requirements=["sentence-transformers", "torch", "transformers"], # If you want to run
)
```

```
# Deploy the model to SPCS
pip_model.create_service(
    service_name="my_minilm_service",
    service_compute_pool="my_gpu_pool", # Using GPU_NV_S - smallest GPU node that can run the
    image_repo="my_image_repo",
    ingress_enabled=True,
    gpu_requests="1", # Model fits in GPU memory; only needed for GPU pool
    max_instances=4, # 4 instances were able to run 10M inferences from an XS warehouse
)
```

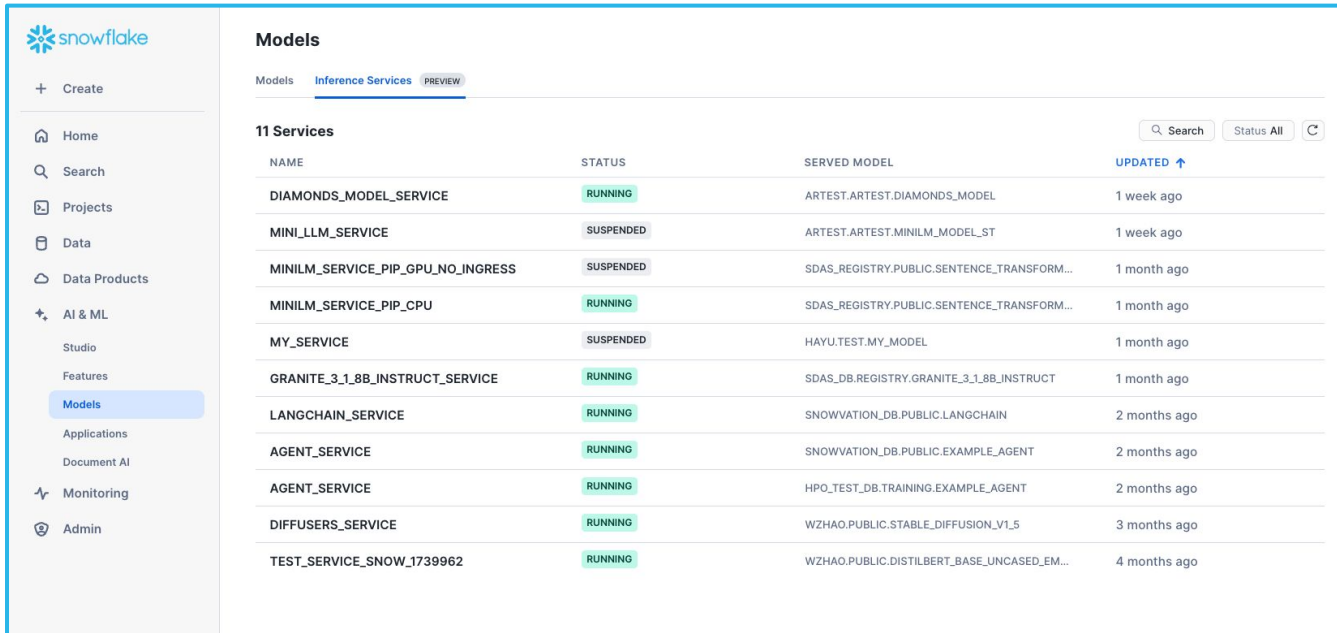
```
# Run on SPCS
pip_model.run(input_data, function_name="encode", service_name="my_minilm_service")
```



Model Serving UI in Snowsight

View model inference services, status, metadata, and logs

UI-based deployment (create service) workflow coming soon!



The screenshot displays the Snowflake Models page in Snowsight. The left sidebar contains navigation options: Create, Home, Search, Projects, Data, Data Products, AI & ML (selected), Studio, Features (with Models highlighted), Applications, Document AI, Monitoring, and Admin. The main content area is titled 'Models' and includes tabs for Models, Inference Services (active), and PREVIEW. It shows 11 services in a table with columns for Name, Status, Served Model, and Updated. The status is color-coded: green for 'RUNNING' and grey for 'SUSPENDED'.

NAME	STATUS	SERVED MODEL	UPDATED ↑
DIAMONDS_MODEL_SERVICE	RUNNING	ARTEST.ARTEST.DIAMONDS_MODEL	1 week ago
MINI_LLM_SERVICE	SUSPENDED	ARTEST.ARTEST.MINILM_MODEL_ST	1 week ago
MINILM_SERVICE_PIP_GPU_NO_INGRESS	SUSPENDED	SDAS_REGISTRY.PUBLIC.SENTENCE_TRANSFORM...	1 month ago
MINILM_SERVICE_PIP_CPU	RUNNING	SDAS_REGISTRY.PUBLIC.SENTENCE_TRANSFORM...	1 month ago
MY_SERVICE	SUSPENDED	HAYU.TEST.MY_MODEL	1 month ago
GRANITE_3_1_8B_INSTRUCT_SERVICE	RUNNING	SDAS_DB.REGISTRY.GRANITE_3_1_8B_INSTRUCT	1 month ago
LANGCHAIN_SERVICE	RUNNING	SNOWVATION_DB.PUBLIC.LANGCHAIN	2 months ago
AGENT_SERVICE	RUNNING	SNOWVATION_DB.PUBLIC.EXAMPLE_AGENT	2 months ago
AGENT_SERVICE	RUNNING	HPO_TEST_DB.TRAINING.EXAMPLE_AGENT	2 months ago
DIFFUSERS_SERVICE	RUNNING	WZHAO.PUBLIC.STABLE_DIFFUSION_V1_5	3 months ago
TEST_SERVICE_SNOW_1739962	RUNNING	WZHAO.PUBLIC.DISTILBERT_BASE_UNCASED_EM...	4 months ago



Partitioned Many-Model Inference

WHAT IS IT

Highly performant and scalable way to run inference in parallel with thousands of models on partitions of data

WHY USE IT

Easily manage and use thousands of individualized models on partitions of data for:

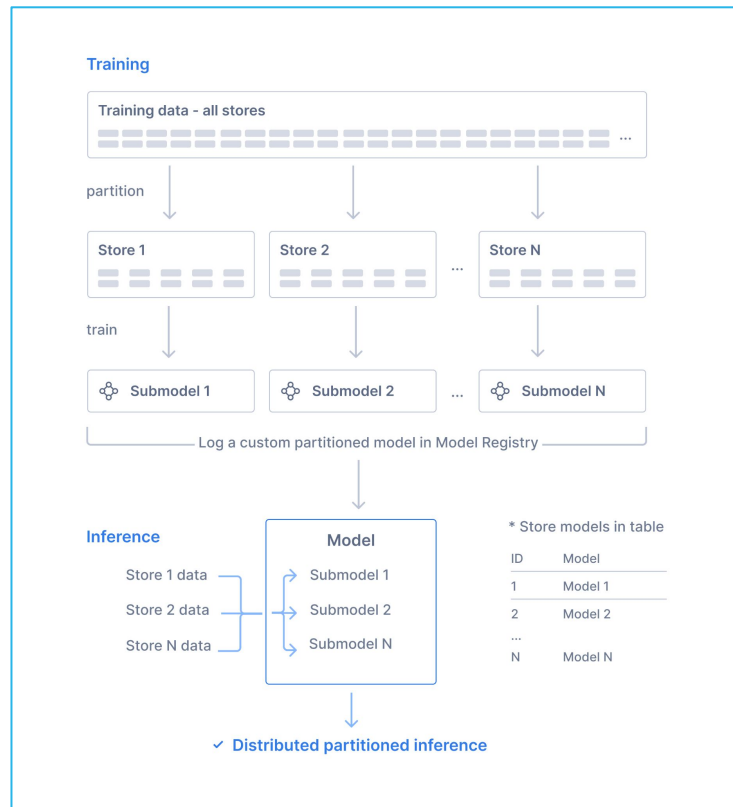
Stateless use cases such as forecasting where model function runs training and inference

Stateful use cases where individual models are stored and used to run batch inference for respective partitions

HOW TO USE IT

Log a single model that references individual models using Custom Model API

Invoke models from Model Registry in parallel in minutes



Partitioned Many-Model Inference

WHAT IS IT

Highly performant and scalable way to run inference in parallel with thousands of models on partitions of data

WHY USE IT

Easily manage and use thousands of individualized models on partitions of data for:

Stateless use cases such as forecasting where model function runs training and inference

Stateful use cases where individual models are stored and used to run batch inference for respective partitions

HOW TO USE IT

Log a single model that references individual models using Custom Model API

Invoke models from Model Registry in parallel in minutes

```
class ExampleForecastingModel(custom_model.CustomModel):

    @custom_model.partitioned_inference_api
    def predict(self, input: pd.DataFrame) -> pd.DataFrame:
        # All data in the partition will be loaded in the input dataframe.
        #... implement model logic here ...
        return output_df

my_model = ExampleForecastingModel()
```

```
reg = Registry(session=sp_session, database_name="ML", schema_name="REGISTRY")
mv = reg.log_model(my_model,
    model_name="my_model",
    version_name="v1",
    options={"function_type": "TABLE_FUNCTION"},    ###
    conda_dependencies=["scikit-learn"],
    sample_input_data=train_features
)
```

```
mv.run(
    input_df,
    function_name="PREDICT",
    partition_column="STORE_NUMBER"
)
```



Model Governance: RBAC

OWNER

- Owner can perform all operations
- Grant USAGE to other roles

```
GRANT USAGE ON MODEL DIAMONDS_MODEL TO ROLE  
OTHER_ROLE;
```

OTHER_ROLE USER

Can

- Perform read-only operations (**SHOW ***) on model versions
- Run inference
`SELECT DIAMONDS_MODEL!predict(...);`

Cannot

- **ALTER**
- See model artifacts



Operationalize Models

Control which version to use in production

Management



Default Version

Update
DEFAULT_VERSION

```
ALTER MODEL MY_MODEL  
SET DEFAULT_VERSION = V3;
```



Using Alias

Manage multiple
labelled versions for
different environments.

```
ALTER MODEL DIAMONDS_MODEL  
VERSION V3 SET ALIAS = 'LIVE';
```



Using TAG

Use TAGs to annotate
versions.

```
ALTER MODEL DIAMONDS_MODEL  
SET TAG LIVE_VERSION = 'V3';
```



Copy Out

Selectively copy model
version from dev to prod
environment.

```
CREATE | ALTER MODEL PROD_MODEL WITH  
| ADD VERSION V1 FROM MODEL  
DEV_MODEL VERSION V12;
```

Inference

```
SELECT  
MY_MODEL!predict(...);
```

```
WITH mv AS MODEL MY_MODEL  
VERSION LIVE  
SELECT mv!predict(...);
```

```
SET LIVE_VER = (SELECT  
SYSTEM$GET_TAG('LIVE_VERSIO  
N', 'DIAMONDS_MODEL',  
'MODULE'));
```

```
WITH mv AS MODEL  
DIAMONDS_MODEL VERSION  
IDENTIFIER($LIVE_VER)  
SELECT mv!predict(...)
```

```
SELECT PROD_MODEL!predict(...);
```



Top 10: Why use Model Registry instead of hand-rolled Python UDFs for ML Models

1. **Do more:** Use GPUs, pip packages, deploy to SPCS, and run online inference with REST API
2. **Monitor models in production:** Monitor model performance and drift
3. **Explain model predictions:** Model explainability built-in for assessing feature contributions
4. **Manage and govern models:** Track versions, metrics, and other metadata; manage model life cycle with aliases and tags
5. Trace flow of data through ML pipelines with **built-in ML Lineage**
6. **Improve security and access:** Fine-grained RBAC with separation of Model Owner and Model Usage roles
7. **Improve user experience:** Snowsight UI for all models; model object is self contained - no need to separately manage python modules and data files from stages;
8. **Improve collaboration:** Models are replicated and can be shared within and across accounts
9. **Bring existing external models to Snowflake,** closer to your data, without any code rewrite
10. **Export models** easily for external use



Snowflake Feature Store Workflows

Snowflake Managed Feature Pipelines

Define features in Snowflake

Minimize end-to-end refresh latency and improve feature freshness with Snowpipe streaming

Automated, continuous feature refresh

Ideal for new features under development

User Managed Feature Pipelines

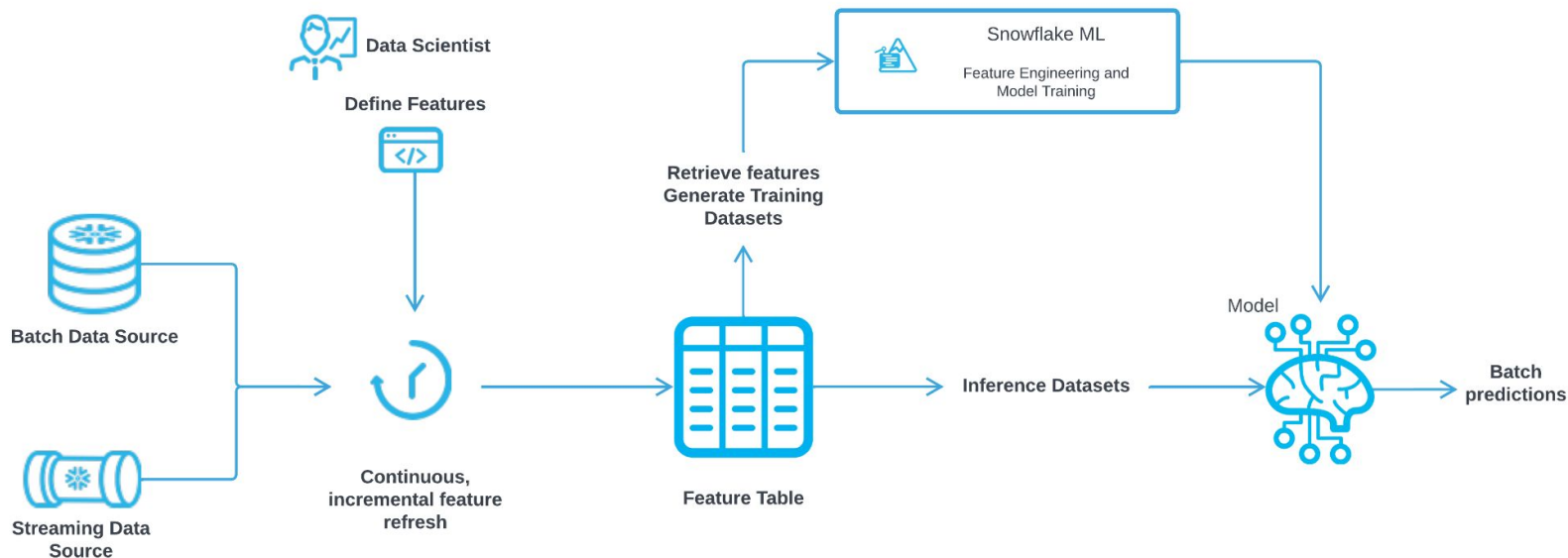
Bring Your Own : For users' with existing "feature tables" managed by their own data-pipelines

- eg. Homegrown, manual Tasks and Streams, DBT, FiveTran, etc

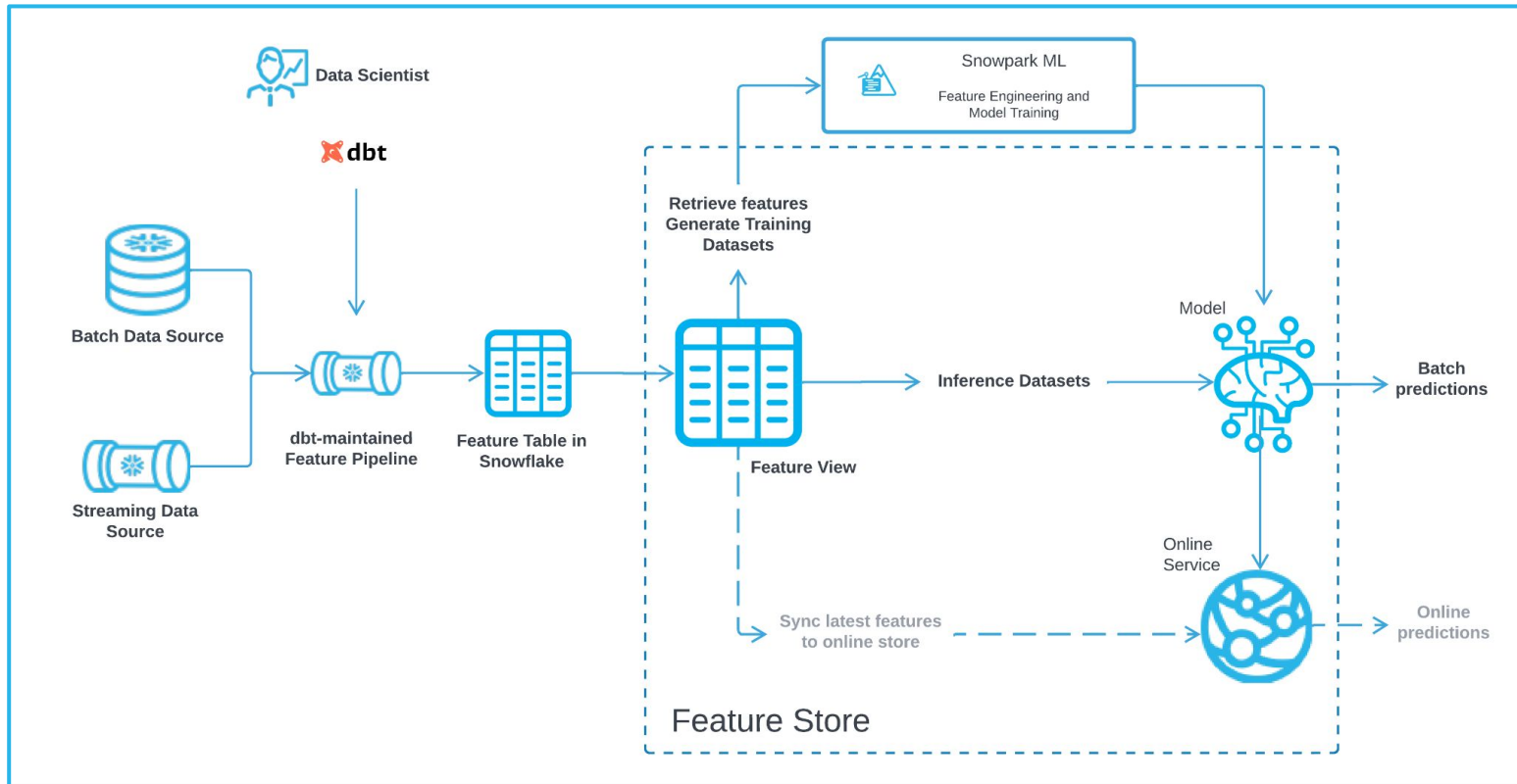
Manage "Features" in Snowflake. Register and govern them in Feature Store for DS/ML applications



Snowflake-Managed Feature Store Workflow

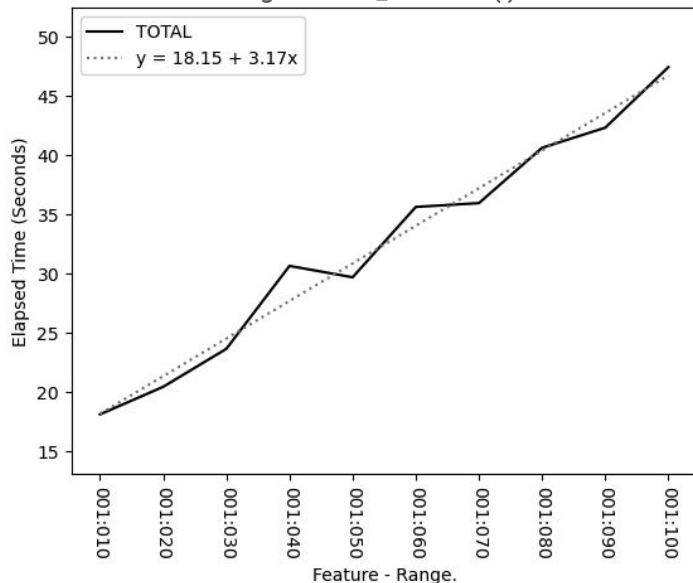


User-Managed Feature : Eg. DBT



Feature Retrieval Scales to Very Large Queries

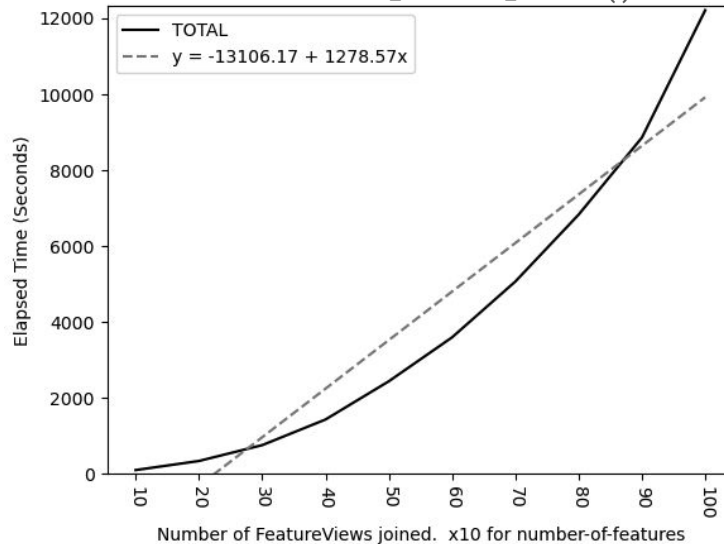
Retrieving training data with
`generate_dataset()`



Elapsed time retrieving feature values with `generate_dataset()`.
Increasing number of features returned from a single FeatureView.

- 10M Entities. 50M records.
- AsOf join over five time-periods per Entity
- From 10 to 100 Features returned.

Retrieving inference data with
`retrieve_feature_values()`



Elapsed time retrieving feature values with
`retrieve_feature_values()`.

Increasing number of Feature Views & Features returned.

- **10 distinct features** per FeatureView.
- 10M Entities. 50M records
- AsOf join over five time-periods per Entity
- From 10 to 100 FeatureViews ASOF joined.

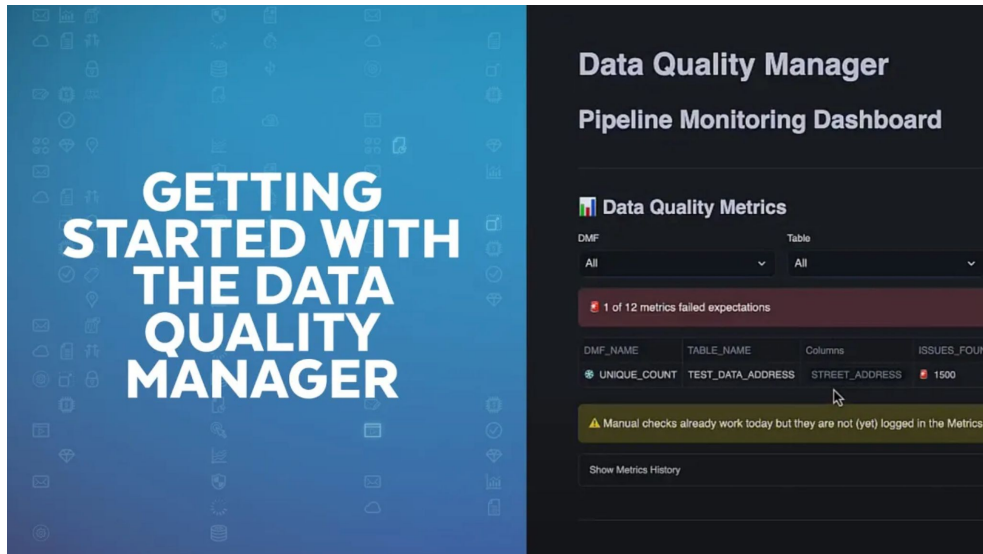
Feature Quality Monitoring

Using Snowflake Horizon Catalog and Data Metric Functions, it is easy to build feature quality monitoring systems

Built-in metrics for null count, missing count, duplicate count, unique count, basic statistics

Define your own custom metrics

Create scheduled monitoring tasks and alerts



[Solution](https://developers.snowflake.com/solution/getting-started-with-the-data-quality-manager/)

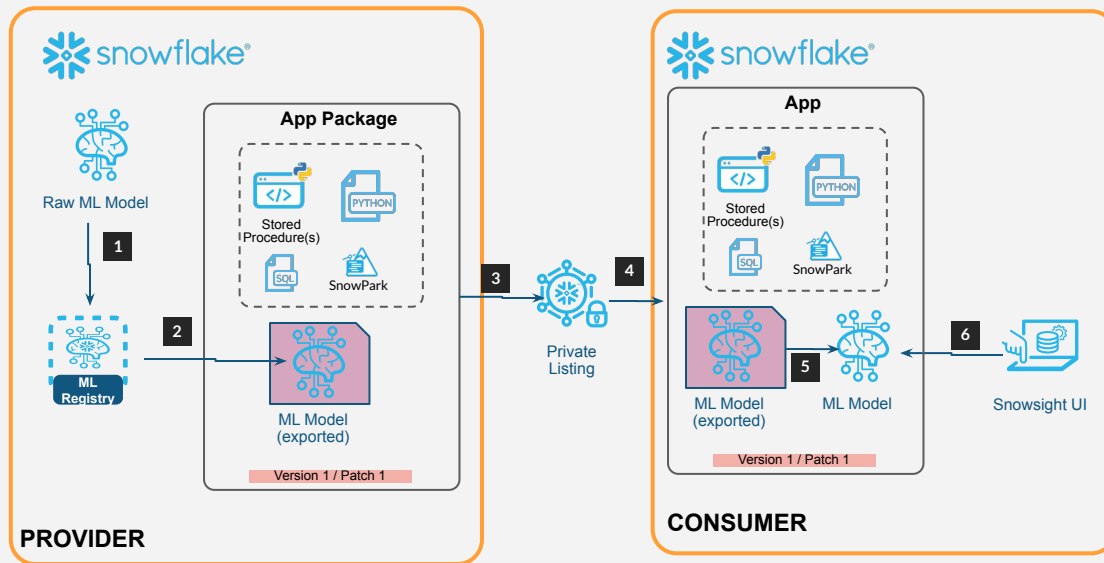
<https://developers.snowflake.com/solution/getting-started-with-the-data-quality-manager/>

[Blog](https://medium.com/snowflake/ensure-data-integrity-with-the-data-quality-manager-6ffeb213134f)

<https://medium.com/snowflake/ensure-data-integrity-with-the-data-quality-manager-6ffeb213134f>



Model Sharing using Native Apps



- 1 - Provider logs / register the ML model, into Snowflake ML registry.
- 2 - Provider exports a specific ML model version and adds it into the application package. (as of today, will change in future).
- 3 - The app is listed in the marketplace.
- 4 - The app is installed in consumer account.
- 5 - During installation or upgrade, a Snowflake ML Model object is created based on the ML model that was exported and packaged. The steps for this is done via SQL statements in setup.sql script.
- 6 - The consumer can invoke specific ML inference function and infer on the unseen data.

[Doc : ML Registry](#)

[Internal Doc : Distributing models in Native Apps](#)

[\(Internal\) Demo shortz : NA Model sharing](#)

NOTES

Currently only support for CPU warehouse based model



Model Sharing using Native Apps

Setup.sql

```
create application role if not exists app_user;  
  
create or alter versioned schema app_code;  
grant usage on schema app_code to application role app_user;  
  
create or replace model app_code.my_model from  
'/models/my_model/versions/V1';  
  
-- grant consumer access to the model if required:  
grant usage on model app_code.my_model to application role  
app_user;
```

Inference from SQL

```
select my_app.app_code.my_model!predict(...);
```



THANK YOU

